Low Level Design Document (LLD)

1. Module and File Structure

The project code is divided into two primary backend services:

- **API Service (`/api`)**: Handles web requests, authentication, file validation, storage uploads, database queries, and job queuing.
- **Worker Service (`/worker`)**: Handles background queue processing, storage downloads, AI API executions, database updates, and notification triggers.

---

2. Database Schema Specifications

The database uses MongoDB with Mongoose object modeling.

2.1 User Model (`models/User.js`)

- `email` (String, required, unique, lowercase, trimmed): The user email address.
- `passwordHash` (String, required): Bcrypt hashed user password.
- `createdAt` (Date, default: Date.now): Account creation timestamp.
- `updatedAt` (Date, default: Date.now): Account last update timestamp.

2.2 Job Model (`models/Job.js`)

- `userId` (ObjectId, ref: User, required, indexed): Owner user ID.
- `originalName` (String, required): Original filename uploaded by user.
- `storagePath` (String, required): Path location inside MinIO/S3 bucket.
- `status` (String, enum: `['pending', 'processing', 'completed', 'failed']`, default: `'pending'`, indexed): Current processing state.
- `caption` (String, optional): Generated natural language description.
- `labels` (Array of Objects `[{ description: String, score: Number }]`, default: `[]`): Detected objects and concepts.
- `safetyResult` (Object with keys `adult`, `spoof`, `medical`, `violence`, `racy`, values enum: `['UNKNOWN', 'VERY_UNLIKELY', 'UNLIKELY', 'POSSIBLE', 'LIKELY', 'VERY_LIKELY']`): Safety classification levels.
- `flagged` (Boolean, default: `false`): Flag indicating if any category is unsafe.
- `flaggedCategories` (Array of Strings, default: `[]`): List of unsafe category names.
- `error` (String, optional): Failure message if processing fails.
- `attempts` (Number, default: `0`): Processing attempt count.
- `createdAt` (Date, default: Date.now): Job creation timestamp.
- `updatedAt` (Date, default: Date.now): Job update timestamp.

2.3 Notification Model (`models/Notification.js`)

- `userId` (ObjectId, ref: User, required, indexed): Recipient user ID.
- `jobId` (ObjectId, ref: Job, required): Associated job ID.
- `message` (String, required): Alert message text.
- `read` (Boolean, default: `false`, indexed): Read status flag.
- `createdAt` (Date, default: Date.now): Alert creation timestamp.

---

3. API Endpoint Technical Specifications

3.1 Authentication Endpoints

- **`POST /api/auth/signup`**

* Auth: None

* Input Body: `{ email: "user@example.com", password: "password123" }`

* Validation: Email and password required, password minimum length 8 characters.

* Success Response: Status 201 Created, returns `{ accessToken, user: { id, email } }` and sets HTTP-only refresh cookie.

- **`POST /api/auth/login`**

* Auth: None

* Input Body: `{ email: "user@example.com", password: "password123" }`

* Validation: Email and password required, bcrypt password match.

* Success Response: Status 200 OK, returns `{ accessToken, user: { id, email } }` and sets HTTP-only refresh cookie.

- **`POST /api/auth/refresh`**

* Auth: Refresh token cookie

* Validation: Validates HTTP-only cookie using JWT secret.

* Success Response: Status 200 OK, returns `{ accessToken }`.

- **`POST /api/auth/logout`**

* Auth: Bearer Token

* Operation: Clears HTTP-only refresh token cookie.

* Success Response: Status 200 OK, returns `{ message: "Logged out" }`.

3.2 File Upload Endpoint

- **`POST /api/upload`**

* Auth: Bearer Token

* Input: Form-data with single file field `image`

* Validation:

1. Multer memory storage filter checks file MIME type against allowed list (`image/jpeg`, `image/png`, `image/webp`).

2. File size limit enforced at 5MB (`5 * 1024 * 1024` bytes).

* Execution Steps:

1. Uploads image buffer to MinIO storage bucket via AWS S3 SDK.

2. Creates new Job record in MongoDB with `status: "pending"`.

3. Adds task to Redis BullMQ queue with `jobId`, `storagePath`, and `userId`.

* Success Response: Status 201 Created, returns `{ jobId: "...", status: "pending" }`.

3.3 Job Query and Retry Endpoints

- **`GET /api/jobs`**

* Auth: Bearer Token

* Query Parameters: `page` (default: 1), `limit` (default: 10)

* Output: Paginated jobs list sorted by creation date descending, enriched with public image URLs.

- **`GET /api/jobs/:id`**

* Auth: Bearer Token

* Path Parameter: `id` (Job ID)

* Output: Single job object with full AI results and image URL.

- **`POST /api/jobs/:id/retry`**

* Auth: Bearer Token

* Path Parameter: `id` (Job ID)

* Validation: Job must exist and have `status: "failed"`.

* Execution Steps: Resets job status to `pending`, clears error string, resets attempts to 0, clears partial results, and re-enqueues job task to BullMQ.

* Success Response: Status 200 OK, returns `{ jobId, status: "pending" }`.

3.4 Notification Endpoints

- **`GET /api/notifications`**: Returns list of user notifications and total unread count.
- **`PUT /api/notifications/read-all`**: Marks all notifications for user as read.
- **`PUT /api/notifications/:id/read`**: Marks a single notification as read.

---

4. Worker Processing Logic and Functions

4.1 Media Processor Routine (`processors/mediaProcessor.js`)

Function `processMedia(jobData)`:

1. Loads Job document from MongoDB by `jobId`.
2. Updates Job status to `processing` and increments `attempts` by 1.
3. Downloads raw image buffer from storage using `downloadFile(storagePath)`.
4. Executes Step 1: `caption = await getCaption(imageBuffer)`.
5. Executes Step 2: `labels = await getLabels(imageBuffer)`.
6. Executes Step 3: `{ safetyResult, flagged, flaggedCategories } = await checkSafety(imageBuffer)`.
7. Updates Job document in MongoDB with `status: "completed"`, `caption`, `labels`, `safetyResult`, `flagged`, `flaggedCategories`.
8. If `flagged` is true, creates a new Notification record in MongoDB for `userId`.

4.2 Captioning Function (`services/captioning.js`)

Function `getCaption(imageBuffer)`:

1. Checks if `process.env.HF_API_KEY` exists.
2. If present, sends HTTP POST request to Hugging Face Inference API (`router.huggingface.co/hf-inference/models/Salesforce/blip-image-captioning-base`) with raw image buffer.
3. If Hugging Face returns HTTP 200 with generated text, returns the text string.
4. If Hugging Face fails or network error occurs, logs a warning and executes fallback.
5. Fallback: Initializes OpenAI client with `process.env.OPENAI_API_KEY`. Converts image buffer to base64 data URL and calls OpenAI `gpt-4o-mini` vision model asking for a one sentence description. Returns the cleaned text.

4.3 Label Detection Function (`services/labelDetection.js`)

Function `getLabels(imageBuffer)`:

1. Converts image buffer to base64 data URL.
2. Calls OpenAI `gpt-4o-mini` chat completion model with prompt requesting main objects and labels in JSON array format `[{ description, score }]`.
3. Parses JSON array from model output using regex pattern match `/\[[\s\S]*\]/`.
4. Maps items to normalized object structure `{ description: String, score: Number }` with score bounded between 0 and 1. Returns array.

4.4 Safety Check Function (`services/safetyCheck.js`)

Function `checkSafety(imageBuffer)`:

1. Converts image buffer to base64 data URL.
2. Calls OpenAI Moderation API model `omni-moderation-latest`.
3. Extracts category scores for `sexual`, `harassment`, `self-harm`, and `violence`.
4. Converts numeric scores (0 to 1) to level strings:

* score >= 0.8: `"VERY_LIKELY"`

* score >= 0.5: `'LIKELY'`

* score >= 0.2: `'POSSIBLE'`

* score >= 0.05: `'UNLIKELY'`

* score < 0.05: `'VERY_UNLIKELY'`

5. Checks if any category returns `'LIKELY'` or `'VERY_LIKELY'`.

6. Sets `flagged = true` if unsafe content is detected and lists flagged categories. Returns `{ safetyResult, flagged, flaggedCategories }`.

---

5. Storage and Queue Infrastructure Specs

5.1 Storage Service (`services/storage.js`)

- Configured with `@aws-sdk/client-s3`.
- Uses `forcePathStyle: true` for MinIO compatibility.
- `uploadFile(buffer, filename)`: Saves file to bucket under unique timestamp key and returns storage path.
- `downloadFile(storagePath)`: Fetches object stream from bucket and converts to Buffer.
- `getImageUrl(storagePath)`: Constructs public HTTP URL for image access.

5.2 Queue Service (`services/queue.js`)

- Configured with `bullmq` Worker and Queue instances backed by `ioredis`.
- Default concurrency set to 2.
- Retry Configuration:

* Automatic retries: Up to 3 attempts with exponential backoff delay (2 seconds, 4 seconds, 8 seconds).

* On job failure after max attempts: BullMQ listener marks Job status as `failed` in MongoDB and saves error message string.